

**МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
КАФЕДРА ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ**

ОТЧЁТ
по лабораторной работе №3
по дисциплине «Основы машинного обучения»
Тема: ПРИМЕНЕНИЕ МЕТОДОВ МАШИННОГО ОБУЧЕНИЯ И
ИХ ОЦЕНКА ДЛЯ ЗАДАЧ КЛАСТЕРИЗАЦИИ

Студент гр. 3312	_____	Барченков П.А.
Студент гр. 3312	_____	Лебедев И.А.
Студент гр. 3312	_____	Шарапов И.Д.
Преподаватель	_____	Петруша П.Г.

Санкт-Петербург
2025

Содержание

ЦЕЛЬ РАБОТЫ	3
ОПИСАНИЕ ЗАДАНИЯ	3
1. Применение кластеризации.....	3
2. Применение алгоритмов регрессии.....	7
ВЫВОДЫ	10
ПРИЛОЖЕНИЕ	11

ЦЕЛЬ РАБОТЫ

Получения и закрепления навыков предобработки данных и применения методов машинного обучения для решения задач кластеризации.

ОПИСАНИЕ ЗАДАНИЯ

Перед началом анализа, необходимо выделить числовые признаки из нашего датасета. Для этого преобразовали столбцы `is_couponed` и `demand` в уникальные числовые значения. Теперь в качестве `numeric_data` будем считать столбцы `total_price`, `listed_price`, `is_couponed`, `reputation` и `demand`.

1. Применение кластеризации

Первым методом кластеризации рассмотрим «метод k-средних». В качестве параметра укажем `n_clusters=9`. Значения основных оценок получились следующими:

- Silhouette Score: 0.6833
- Davies-Bouldin Index: 0.4514

Визуализируем полученное разделение (рис. 1):

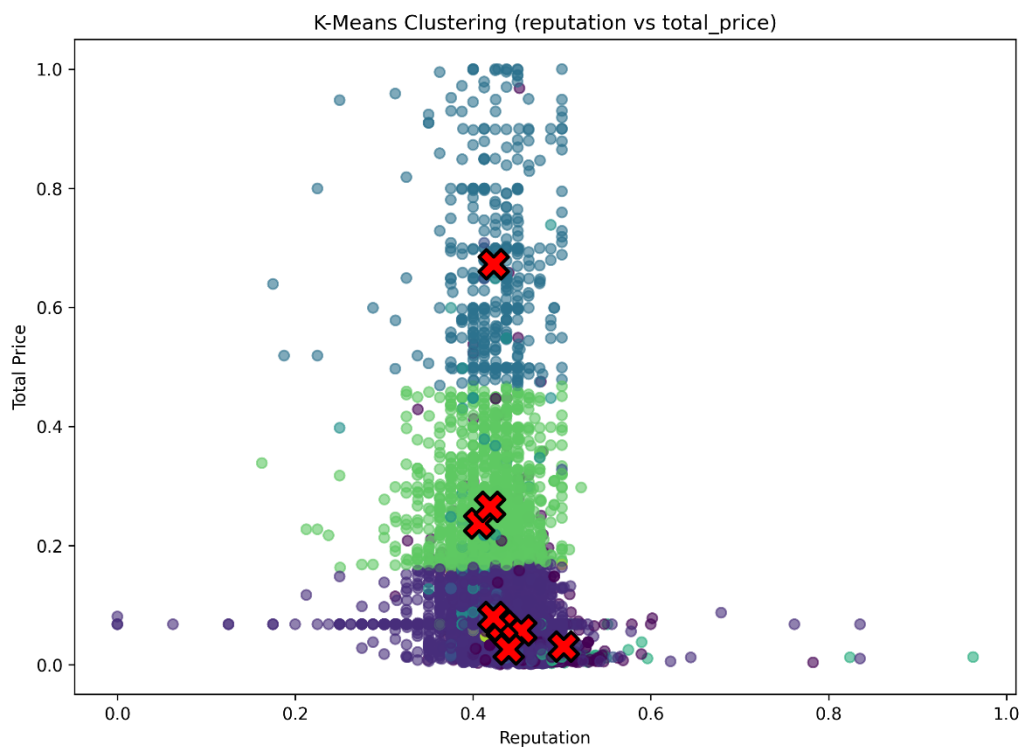


Рисунок 1 – Метод k-средних (`n_clusters=9`)

Несмотря на то, что алгоритм искал 9 кластеров, на графике хорошо выделяются именно 3 кластера. Попробуем поменять параметр `n_clusters=3` (рис. 2):

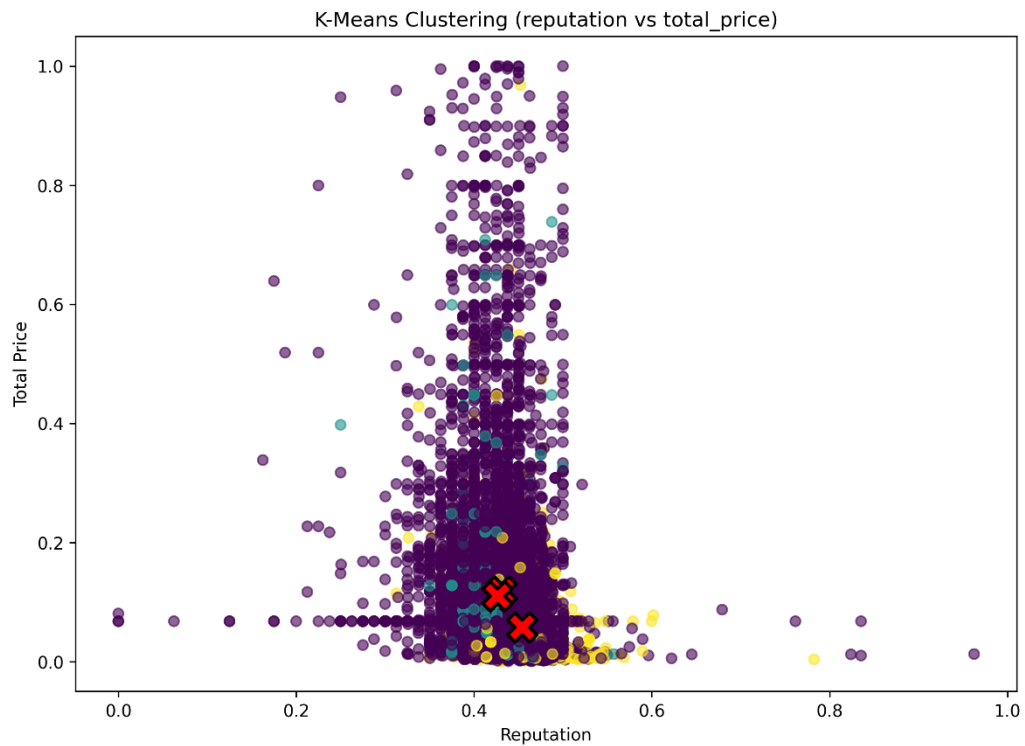


Рисунок 3 – Метод k-средних (`n_clusters=3`)

Теперь чётко виден только 1 кластер. Вернёмся к 9 кластерам.

Следующим методом кластеризации рассмотрим «иерархическую кластеризацию». В качестве параметров `n_clusters=9`, `linkage=linkage_method`. Значения основных оценок получились следующими:

- Silhouette Score: 0.7201
- Davies-Bouldin Index: 0.4680

Построим дендограмму (рис. 3):

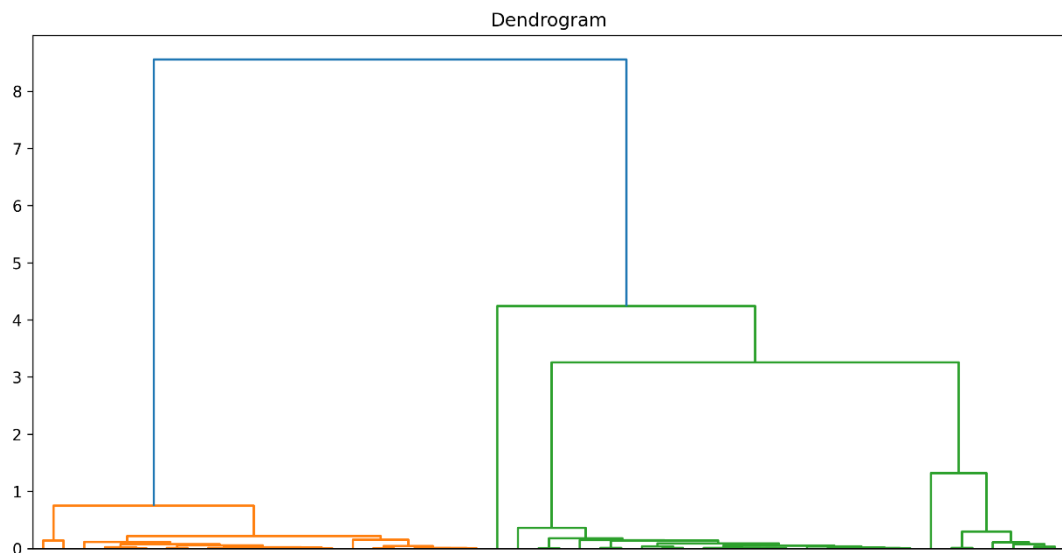


Рисунок 3 – Дендрограмма

Рассмотрим диаграмму рассеяния по total_price и reputation (рис. 4):

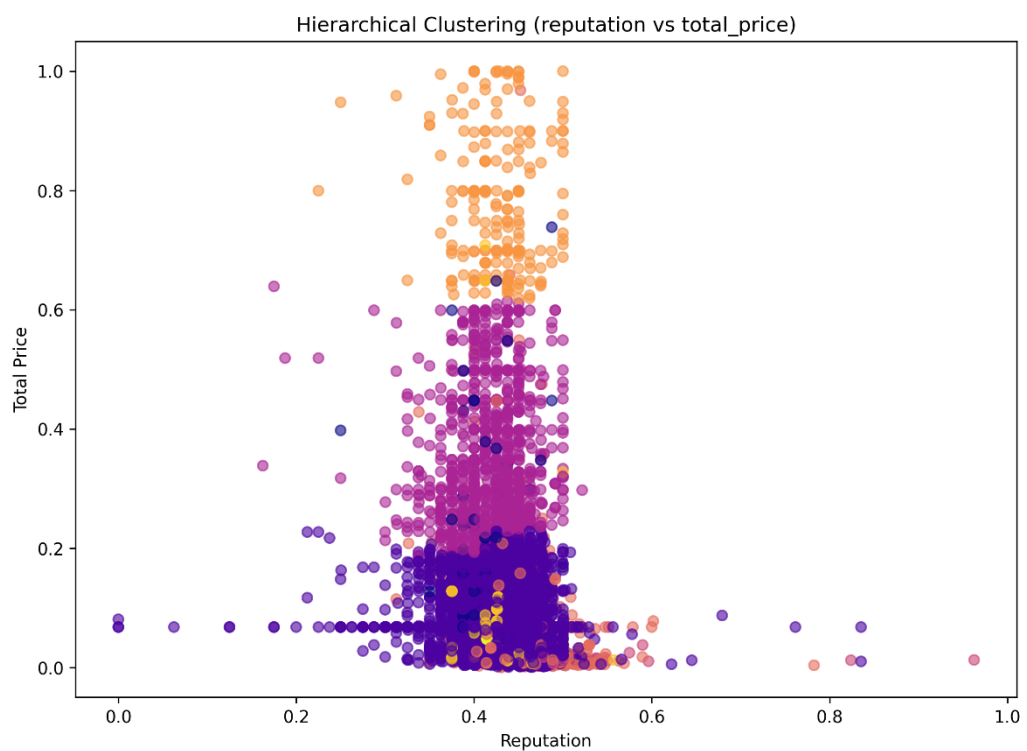


Рисунок 4 – Метод иерархической кластеризации

Аналогично предыдущему методу на рисунке чётко выделяются 3 кластера.

Следующим методом кластеризации рассмотрим «DBSCAN». В качестве параметра $\text{eps}=0.9$. Значения основных оценок получились следующими:

- Number of clusters: 9
- Number of noise points: 4
- Silhouette Score: 0.8650

Визуализируем полученное разделение (рис. 5):

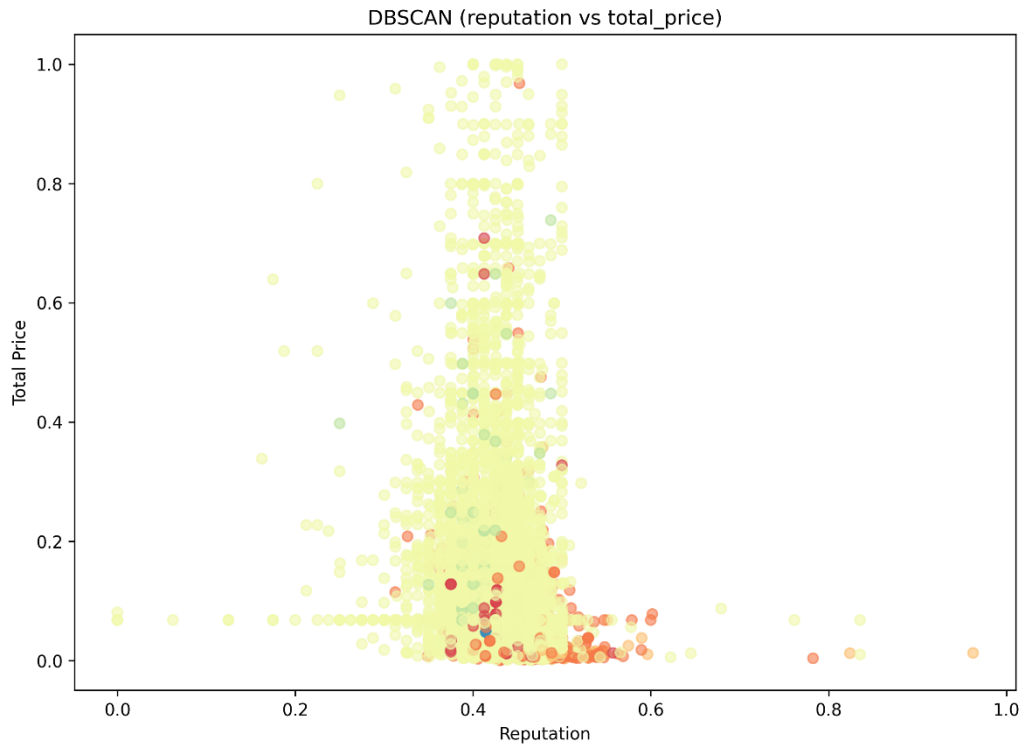


Рисунок 5 – Метод DBSCAN

Несмотря на то, что алгоритм выделил также 9 кластеров, на графике можно чётко выделить только 1 кластер.

Для всех трёх методов оптимальным оказалось значение в 9 кластеров. При этом лучший показатель Silhouette Score показал метод DBSCAN, а Davies-Bouldin Index у иерархической кластеризации.

2. Применение алгоритмов регрессии

В качестве предсказываемого параметра был выбран столбец `total_price`. Для его прогноза рассматриваются 3 параметра `is_couponed`, `reputation` и `demand`.

Применим метод «линейной регрессии» и рассмотрим её основные показатели (рис. 6):

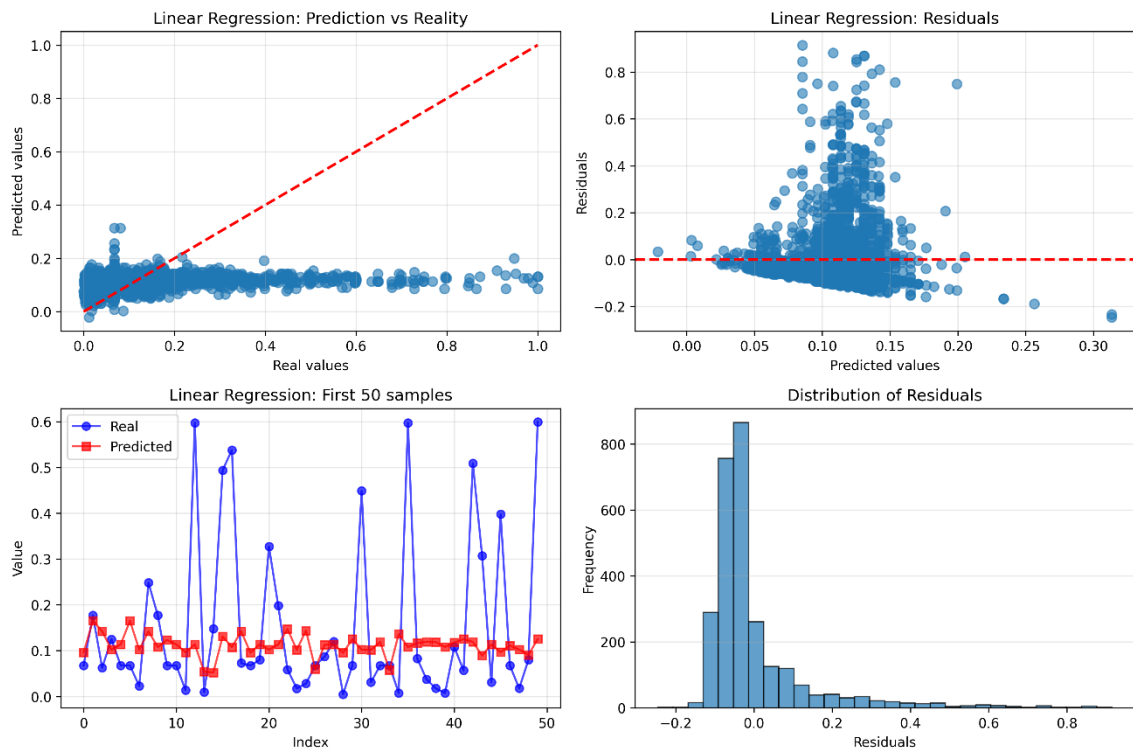


Рисунок 6 – Показатели для линейной регрессии

Численные параметры:

- MSE: 0.018658
- RMSE: 0.136595
- MAE: 0.086179
- R^2 : 0.036980

Проведём аналогичный эксперимент для метода «LASSO» (рис. 7):

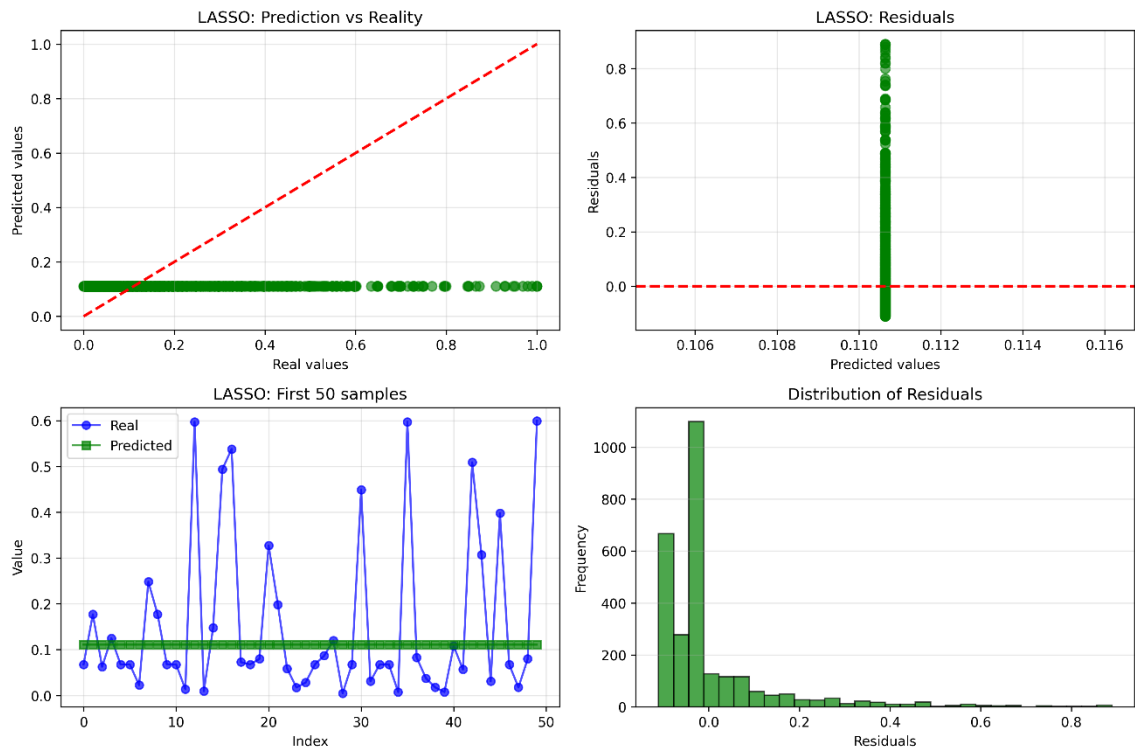


Рисунок 7 – Показатели для LASSO

Численные параметры:

- MSE: 0.019375
- RMSE: 0.139194
- MAE: 0.088964
- R^2 : -0.000021

А также для «гребневой (Риндж) регрессии» (рис. 8):

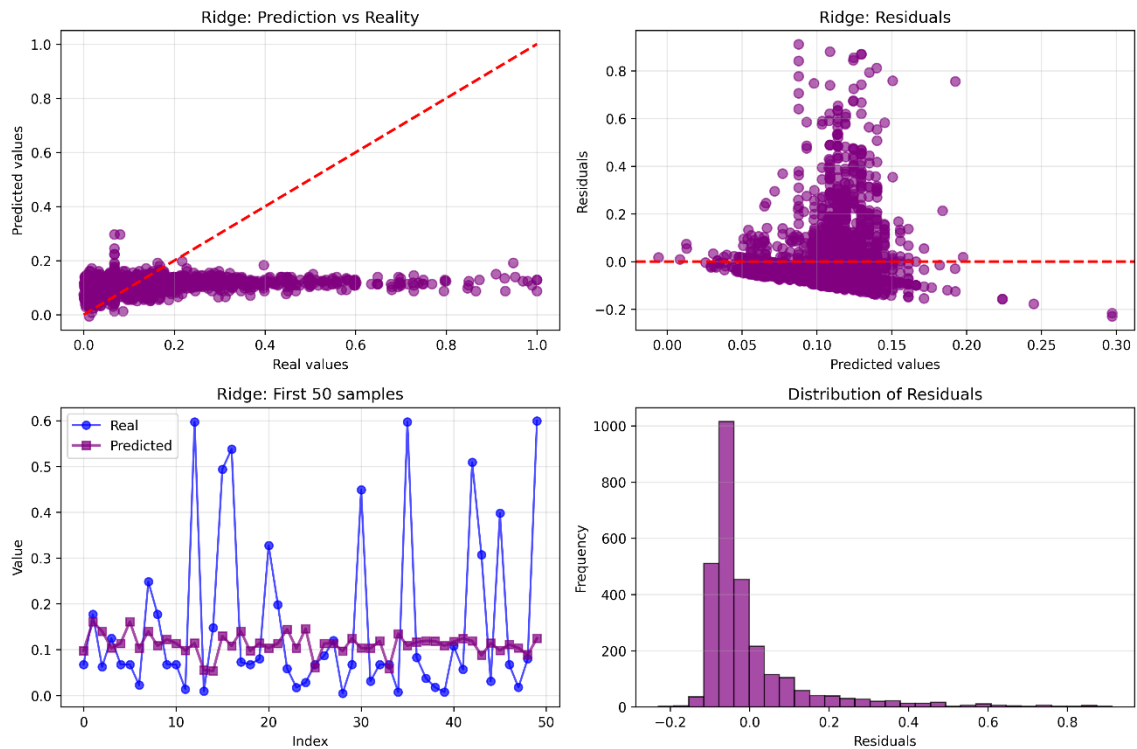


Рисунок 8 – Показатели для Ридж регрессии

Численные параметры:

- MSE: 0.018663
- RMSE: 0.136611
- MAE: 0.086226
- R^2 : 0.036745

Линейная и Ридж регрессия очень похожи по полученным результатам и работают нормально на данном датасете. В свою очередь LASSO показывает очень плохие результаты и практически неприменим в данном датасете.

ВЫВОДЫ

В ходе работы были применены три основных метода кластеризации: K-Means, иерархическая кластеризация и DBSCAN. Все методы были использованы с параметром `n_clusters=9`.

При сравнении результатов кластеризации выявлено, что метод DBSCAN показал лучший результат по метрике Silhouette Score. Иерархическая кластеризация также показала хороший. K-Means показал немного более низкие результаты, однако также продемонстрировал адекватную кластеризацию данных.

Визуально кластеры лучше всего различимы в методе k-средних, когда у DBSCAN почти всё окрашено в один цвет.

Для регрессии была выбрана целевая переменная `total_price`, которую требовалось предсказать по трём признакам: `is_couponed`, `reputation` и `demand`. Признак `listed_price` был исключён из анализа из-за высокой корреляции с целевой переменной.

Линейная и Ridge регрессии показали практически идентичные результаты что указывает на то, что регуляризация практически не повлияла на качество модели. LASSO регрессия показала худшие результаты, что свидетельствует о чрезмерной регуляризации и обнулении важных коэффициентов.

ПРИЛОЖЕНИЕ

Код программы

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
from sklearn.linear_model import LinearRegression, Lasso, Ridge
from sklearn.metrics import (
    silhouette_score, davies_bouldin_score, calinski_harabasz_score,
    mean_squared_error, r2_score, mean_absolute_error)
from sklearn.preprocessing import LabelEncoder
from scipy.cluster.hierarchy import dendrogram, linkage

data = pd.read_csv('data/amazon_products_sales_data_cleaned.csv')

print(f"Data size: {data.shape}")
print(data.head().to_string())

data_processed = data.copy()
le_coupon = LabelEncoder()
data_processed['is_couponed'] =
le_coupon.fit_transform(data_processed['is_couponed'].astype(str))
le_demand = LabelEncoder()
data_processed['demand'] = le_demand.fit_transform(data_processed['demand'].astype(str))
columns_to_keep = ['total_price', 'listed_price', 'is_couponed', 'reputation', 'demand']
numeric_data = data_processed[columns_to_keep].dropna()
X = numeric_data.values

print("\n=== K-MEANS CLUSTERING ===")
kmeans = KMeans(n_clusters=9, random_state=42, n_init=10)
kmeans_labels = kmeans.fit_predict(X)

kmeans_silhouette = silhouette_score(X, kmeans_labels)
kmeans_davies_bouldin = davies_bouldin_score(X, kmeans_labels)
kmeans_calinski = calinski_harabasz_score(X, kmeans_labels)
print(f"Silhouette Score: {kmeans_silhouette:.4f}")
print(f"Davies-Bouldin Index: {kmeans_davies_bouldin:.4f}")

plt.figure(figsize=(10, 7))
plt.scatter(numeric_data['reputation'], numeric_data['total_price'], c=kmeans_labels,
            cmap='viridis', alpha=0.6)
plt.scatter(kmeans.cluster_centers[:, numeric_data.columns.get_loc('reputation')],
            kmeans.cluster_centers[:, numeric_data.columns.get_loc('total_price')], c='red',
            marker='X', s=300, edgecolors='black', linewidth=2)
plt.xlabel('Reputation')
plt.ylabel('Total Price')
plt.title('K-Means Clustering (reputation vs total price)')
plt.savefig('images/01_kmeans.png', dpi=300, bbox_inches='tight')
plt.close()

print("\n=== HIERARCHICAL CLUSTERING ===")
linkage_method = 'ward'
agglom = AgglomerativeClustering(n_clusters=9, linkage=linkage_method)
agglom_labels = agglom.fit_predict(X)

agglom_silhouette = silhouette_score(X, agglom_labels)
agglom_davies_bouldin = davies_bouldin_score(X, agglom_labels)
agglom_calinski = calinski_harabasz_score(X, agglom_labels)
print(f"Silhouette Score: {agglom_silhouette:.4f}")
print(f"Davies-Bouldin Index: {agglom_davies_bouldin:.4f}")

# Дендрограмма
Z = linkage(X[:min(50, len(X))], method=linkage_method)
plt.figure(figsize=(12, 6))
```

```

dendrogram(Z, no_labels=True)
plt.title(f'Dendrogram')
plt.savefig('images/02_dendrogram.png', dpi=300, bbox_inches='tight')
plt.close()

plt.figure(figsize=(10, 7))
plt.scatter(numeric_data['reputation'], numeric_data['total_price'], c=agglom_labels,
            cmap='plasma', alpha=0.6)
plt.xlabel('Reputation')
plt.ylabel('Total Price')
plt.title('Hierarchical Clustering (reputation vs total price)')
plt.savefig('images/03_hierarchical.png', dpi=300, bbox_inches='tight')
plt.close()

print("\n== DBSCAN CLUSTERING ==")
dbscan = DBSCAN(eps=0.9, min_samples=5)
dbscan_labels = dbscan.fit_predict(X)

n_clusters_dbscan = len(set(dbscan_labels)) - (1 if -1 in dbscan_labels else 0)
n_noise = list(dbscan_labels).count(-1)
print(f"Number of clusters: {n_clusters_dbscan}")
print(f"Number of noise points: {n_noise}")
if n_clusters_dbscan > 1:
    dbscan_silhouette = silhouette_score(X[dbscan_labels != -1],
                                         dbscan_labels[dbscan_labels != -1])
    print(f"Silhouette Score: {dbscan_silhouette:.4f}")

plt.figure(figsize=(10, 7))
plt.scatter(numeric_data['reputation'], numeric_data['total_price'],
            c=dbscan_labels, cmap='Spectral', alpha=0.6)
plt.xlabel('Reputation')
plt.ylabel('Total Price')
plt.title('DBSCAN (reputation vs total price)')
plt.savefig('images/04_dbscan.png', dpi=300, bbox_inches='tight')
plt.close()

target_col = 'total_price'
X_reg = numeric_data.drop(columns=['total_price', 'listed_price']).values
y_reg = numeric_data[target_col].values
X_train, X_test, y_train, y_test = train_test_split(X_reg, y_reg, test_size=0.3,
                                                    random_state=42)

print("\n== LINEAR REGRESSION ==")
lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
mse_lr = mean_squared_error(y_test, y_pred_lr)
rmse_lr = np.sqrt(mse_lr)
r2_lr = r2_score(y_test, y_pred_lr)
mae_lr = mean_absolute_error(y_test, y_pred_lr)
print(f"MSE: {mse_lr:.6f}, RMSE: {rmse_lr:.6f}, MAE: {mae_lr:.6f}, R²: {r2_lr:.6f}")

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
axes[0, 0].scatter(y_test, y_pred_lr, alpha=0.6, s=50)
axes[0, 0].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
axes[0, 0].set_xlabel('Real values')
axes[0, 0].set_ylabel('Predicted values')
axes[0, 0].set_title('Linear Regression: Prediction vs Reality')
axes[0, 0].grid(True, alpha=0.3)
residuals_lr = y_test - y_pred_lr
axes[0, 1].scatter(y_pred_lr, residuals_lr, alpha=0.6, s=50)
axes[0, 1].axhline(y=0, color='r', linestyle='--', lw=2)
axes[0, 1].set_xlabel('Predicted values')
axes[0, 1].set_ylabel('Residuals')
axes[0, 1].set_title('Linear Regression: Residuals')
axes[0, 1].grid(True, alpha=0.3)
axes[1, 0].plot(y_test[:50], 'b-o', label='Real', alpha=0.7)

```

```

axes[1, 0].plot(y_pred_lr[:50], 'r-s', label='Predicted', alpha=0.7)
axes[1, 0].set_xlabel('Index')
axes[1, 0].set_ylabel('Value')
axes[1, 0].set_title('Linear Regression: First 50 samples')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)
axes[1, 1].hist(residuals_lr, bins=30, edgecolor='black', alpha=0.7)
axes[1, 1].set_xlabel('Residuals')
axes[1, 1].set_ylabel('Frequency')
axes[1, 1].set_title('Distribution of Residuals')
axes[1, 1].grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.savefig('images/05_linear_regression.png', dpi=300, bbox_inches='tight')
plt.close()

print("\n== LASSO REGRESSION ==")
lasso = Lasso()
lasso.fit(X_train, y_train)
y_pred_lasso = lasso.predict(X_test)
mse_lasso = mean_squared_error(y_test, y_pred_lasso)
rmse_lasso = np.sqrt(mse_lasso)
r2_lasso = r2_score(y_test, y_pred_lasso)
mae_lasso = mean_absolute_error(y_test, y_pred_lasso)
print(f"MSE: {mse_lasso:.6f}, RMSE: {rmse_lasso:.6f}, MAE: {mae_lasso:.6f}, R²: {r2_lasso:.6f}")

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
axes[0, 0].scatter(y_test, y_pred_lasso, alpha=0.6, s=50, color='green')
axes[0, 0].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
axes[0, 0].set_xlabel('Real values')
axes[0, 0].set_ylabel('Predicted values')
axes[0, 0].set_title('LASSO: Prediction vs Reality')
axes[0, 0].grid(True, alpha=0.3)
residuals_lasso = y_test - y_pred_lasso
axes[0, 1].scatter(y_pred_lasso, residuals_lasso, alpha=0.6, s=50, color='green')
axes[0, 1].axhline(y=0, color='r', linestyle='--', lw=2)
axes[0, 1].set_xlabel('Predicted values')
axes[0, 1].set_ylabel('Residuals')
axes[0, 1].set_title('LASSO: Residuals')
axes[0, 1].grid(True, alpha=0.3)
axes[1, 0].plot(y_test[:50], 'b-o', label='Real', alpha=0.7)
axes[1, 0].plot(y_pred_lasso[:50], 'g-s', label='Predicted', alpha=0.7)
axes[1, 0].set_xlabel('Index')
axes[1, 0].set_ylabel('Value')
axes[1, 0].set_title('LASSO: First 50 samples')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)
axes[1, 1].hist(residuals_lasso, bins=30, edgecolor='black', alpha=0.7, color='green')
axes[1, 1].set_xlabel('Residuals')
axes[1, 1].set_ylabel('Frequency')
axes[1, 1].set_title('Distribution of Residuals')
axes[1, 1].grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.savefig('images/06_lasso_regression.png', dpi=300, bbox_inches='tight')
plt.close()

print("\n== RIDGE REGRESSION ==")
ridge = Ridge()
ridge.fit(X_train, y_train)
y_pred_ridge = ridge.predict(X_test)
mse_ridge = mean_squared_error(y_test, y_pred_ridge)
rmse_ridge = np.sqrt(mse_ridge)
r2_ridge = r2_score(y_test, y_pred_ridge)
mae_ridge = mean_absolute_error(y_test, y_pred_ridge)
print(f"MSE: {mse_ridge:.6f}, RMSE: {rmse_ridge:.6f}, MAE: {mae_ridge:.6f}, R²: {r2_ridge:.6f}")

fig, axes = plt.subplots(2, 2, figsize=(12, 8))

```

```

axes[0, 0].scatter(y_test, y_pred_ridge, alpha=0.6, s=50, color='purple')
axes[0, 0].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
axes[0, 0].set_xlabel('Real values')
axes[0, 0].set_ylabel('Predicted values')
axes[0, 0].set_title('Ridge: Prediction vs Reality')
axes[0, 0].grid(True, alpha=0.3)
residuals_ridge = y_test - y_pred_ridge
axes[0, 1].scatter(y_pred_ridge, residuals_ridge, alpha=0.6, s=50, color='purple')
axes[0, 1].axhline(y=0, color='r', linestyle='--', lw=2)
axes[0, 1].set_xlabel('Predicted values')
axes[0, 1].set_ylabel('Residuals')
axes[0, 1].set_title('Ridge: Residuals')
axes[0, 1].grid(True, alpha=0.3)
axes[1, 0].plot(y_test[:50], 'b-o', label='Real', alpha=0.7)
axes[1, 0].plot(y_pred_ridge[:50], 'purple', marker='s', label='Predicted', linewidth=2,
markersize=6, alpha=0.7, linestyle='-')
axes[1, 0].set_xlabel('Index')
axes[1, 0].set_ylabel('Value')
axes[1, 0].set_title('Ridge: First 50 samples')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)
axes[1, 1].hist(residuals_ridge, bins=30, edgecolor='black', alpha=0.7, color='purple')
axes[1, 1].set_xlabel('Residuals')
axes[1, 1].set_ylabel('Frequency')
axes[1, 1].set_title('Distribution of Residuals')
axes[1, 1].grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.savefig('images/07 ridge regression.png', dpi=300, bbox_inches='tight')
plt.close()

```

Ссылка на GitHub: https://github.com/DexTver/ML_LETI/tree/lab_03